Generative Data Generation with PCA and GMM

AI53: Machine Learning by Practice

Work Session

1 The Problem of Data Generation

In the era of data-driven science and machine learning, data generation has emerged as a critical capability with numerous applications:

- Data Augmentation: When training machine learning models, especially deep neural networks, we often need more data than available. Synthetic data can augment real datasets, improving model generalization and preventing overfitting.
- Privacy Preservation: In healthcare, finance, and other sensitive domains, synthetic data can mimic real data patterns without exposing actual private information.
- Rare Event Simulation: For anomaly detection or risk assessment, we need examples of rare events that are scarce in real data.
- Algorithm Testing: Researchers and engineers need controlled datasets to test new algorithms under various conditions.
- System Design: In engineering, synthetic data helps design systems before real data becomes available.

Key Concept

The Core Challenge: Real data often lies on a complex, high-dimensional manifold. The goal of generative modeling is to learn this manifold structure and sample new points from it that are statistically similar to the original data.

2 Overview of the Work Session

In this work session, you will implement a complete pipeline for generating synthetic data using a combination of dimensionality reduction and probabilistic modeling. Here's the conceptual flow:

1. High-Dimensional Data: We start with simulated Gaussian curves, which are continuous functions. Even after discretization (sampling at `nbx` points), each curve lives in a `nbx`-dimensional space.
2. Dimensionality Reduction with PCA: Gaussian curves aren't truly `nbx`-dimensional; they're parameterized by just two numbers: the mean μ and the standard deviation σ. Principal Component Analysis (PCA) will help us discover this intrinsic low-dimensional structure.
3. Density Estimation with GMM: In the reduced PCA space, we'll model the data distribution using a Gaussian Mixture Model (GMM). This gives us a probabilistic model of where the data points are concentrated.

4. Data Generation: Finally, we'll sample new points from the GMM distribution and map them back to the original high-dimensional space, generating new Gaussian curves that follow the same statistical patterns as our original data.

Mathematical Background

The Mathematical Pipeline:

$$\begin{aligned}
&\text{High-dim data } X \in \mathbb{R}^{N\times p}\\
&\text{PCA: } Z = (X-\bar{X})V_k \in \mathbb{R}^{N\times d} \quad (d \ll p)\\
&\text{GMM: } p(z) = \sum_{i=1}^{K} \pi_i \mathcal{N}(z|\mu_i, \Sigma_i)\\
&\text{Sample: } z_{\text{new}} \sim p(z)\\
&\text{Reconstruct: } x_{\text{new}} = z_{\text{new}} V_k^T + \bar{X}
\end{aligned}$$

This approach demonstrates a fundamental principle in machine learning: many high-dimensional datasets have much lower intrinsic dimensionality, and modeling in this reduced space is both more efficient and more effective.

3 Step-by-Step Implementation

3.1 Generating Gaussian Curves

We begin by creating our "ground truth" dataset: Gaussian curves with varying parameters. Each curve is defined by:

$$x \to f(x|\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

and for each values of (μ, σ), we get a different curve.

This part is given in the associated notebook.

3.2 Principal Component Analysis

PCA finds the directions of maximum variance in the data. For centered data $X_c = X - \bar{X}$, we compute:

$$C = \frac{1}{N-1} X_c^T X_c \quad \text{(covariance matrix)}$$

The eigenvectors of C are the loadings (components axis), and the eigenvalues represent the variance explained.

This part is given in the associated notebook.

3.3 Step 3: Gaussian Mixture Model Clustering

In the reduced PCA space, we model the data distribution as a mixture of Gaussian distributions:

$$p(z) = \sum_{i=1}^{K} \pi_i \mathcal{N}(z|\mu_i, \Sigma_i)$$

where π_i are mixing coefficients ($\sum_i \pi_i = 1$).

```python
def cluster_with_gmm(Z, n_clusters):
    """
    Cluster data in PCA space using Gaussian Mixture Model.

    Parameters:
    Z : array of shape (N, n_components) - data in PCA space
    n_clusters : number of GMM components

    Returns:
    Dictionary (or list) containing:
    - gmm : fitted GMM object
    - cluster_labels : cluster assignments for each point
    - cluster_probs : probability of belonging to each cluster
    """
    # Your implementation here
    # Use sklearn.mixture.GaussianMixture
    # Return the model and cluster assignments
```

Listing 1: GMM clustering function

Important Considerations:

- Visualize clusters in the first 2-3 PCA dimensions
- Analyze what types of Gaussian curves belong to each cluster. Do they behave differently?

3.4 Step 4: Sampling New Data

Sample from the learned GMM distribution and reconstruct new curves:

1. Sample a cluster k with probability π_k
2. Sample $z_{\text{new}} \sim \mathcal{N}(\mu_k, \Sigma_k)$
3. Reconstruct: $x_{\text{new}} = z_{\text{new}} V_k^T + \bar{X}$

```python
def sample_new_curves(gmm, pca, scaler, n_samples=100):
    """
    Sample new curves from the learned GMM distribution.

    Parameters:
    gmm : fitted GMM object
    pca : fitted PCA object
    scaler : fitted StandardScaler object
    n_samples : number of curves to generate

    Returns:
    Dictionary [or list] containing:
    - samples_pca : samples in PCA space
    - samples_original : reconstructed curves
    - sample_clusters : cluster assignments for samples
    """
    # Your implementation here
    # Use gmm.sample() to sample from GMM
    # Use pca.inverse_transform() to reconstruct
    # Don't forget to invert scaling
```

Listing 2: Data sampling function

4 Experiments and Analysis

After implementing the complete pipeline, conduct systematic experiments to understand how different choices affect the results.

4.1 Experiment 1: Varying PCA Components

Test how the number of PCA components affects:

- Reconstruction error of original curves
- Quality of generated curves
- Computational efficiency

```
    for n_components in [2, 5, 10, 20, 50,....]:
        # Run PCA with n_components
        # Generate new curves
        # Compare with original statistics
```

4.2 Experiment 2: Varying GMM Clusters

Test how the number of clusters affects:

- Cluster separation and interpretability
- Diversity of generated curves

```
    for n_clusters in [2, 3, 5, 8, 10, 15,....]:
        # Run GMM with n_clusters
        # Analyze cluster statistics
        # Generate and compare curves
```

4.3 Experiment 3: Statistical Comparison

Quantitatively compare original and generated curves:

- Mean and standard deviation profiles
- Distribution of curve parameters (height, width, position)
- Proportion of negative values (True Gaussian curves are always positive)

```
    # Calculate various similarity metrics:
    # 1. Mean squared error of mean curves
    # 2. Distribution of curve parameters
    # 3. Proportion of negative values
```

4.4 Experiment 4: Robustness Analysis

Test the pipeline's robustness to:

- Different noise levels in original data
- Different parameter distributions for μ and σ

5 Submission Requirements

What to Submit

1. **Jupyter Notebook (`main_notebook.ipynb`)**: A well-organized notebook containing:
   - Clear section headers and explanations in markdown cells
   - All experiments and visualizations
   - Analysis and interpretation of results
   - Answers to all questions posed in this guide
2. **Python Modules** - Clean, commented Python files:
   - `create_gaussian_curves.py` - Gaussian curve generation
   - `analyse_pca.py` - PCA implementation
   - `gmm_clustering.py` - GMM clustering
   - `curve_sampling.py` - Sampling new curves
   - `visualization.py` - Plotting functions (optional but recommended)
3. **Short Report (Markdown cells)** - Add Markdown cell(s) with
   - Your approach and implementation decisions
   - Key findings from experiments
   - Answers to the reflection questions below (optional)

Reflection Questions

In your report, you can address the following questions (list not limited):

1. Dimensionality Reduction: How many PCA components were needed to adequately represent your Gaussian curves? Why do you think this number makes sense given how the curves were generated?
2. Generation Quality: How realistic are the generated curves compared to the original ones? What statistical properties are preserved well, and which ones are not?
3. Pipeline Limitations: What are the main limitations of this PCA+GMM approach for data generation? When would it work well, and when might it fail?
4. Practical Applications: Based on your experience, suggest one or two real-world applications where this approach could be useful. What modifications might be needed for those applications?

Deadline and Submission

- **Due Date**: 09/01/2026
- **File Format**: Compressed archive containing all files
- **Naming Convention**: `LASTNAME_FirstName_Work Session.zip`